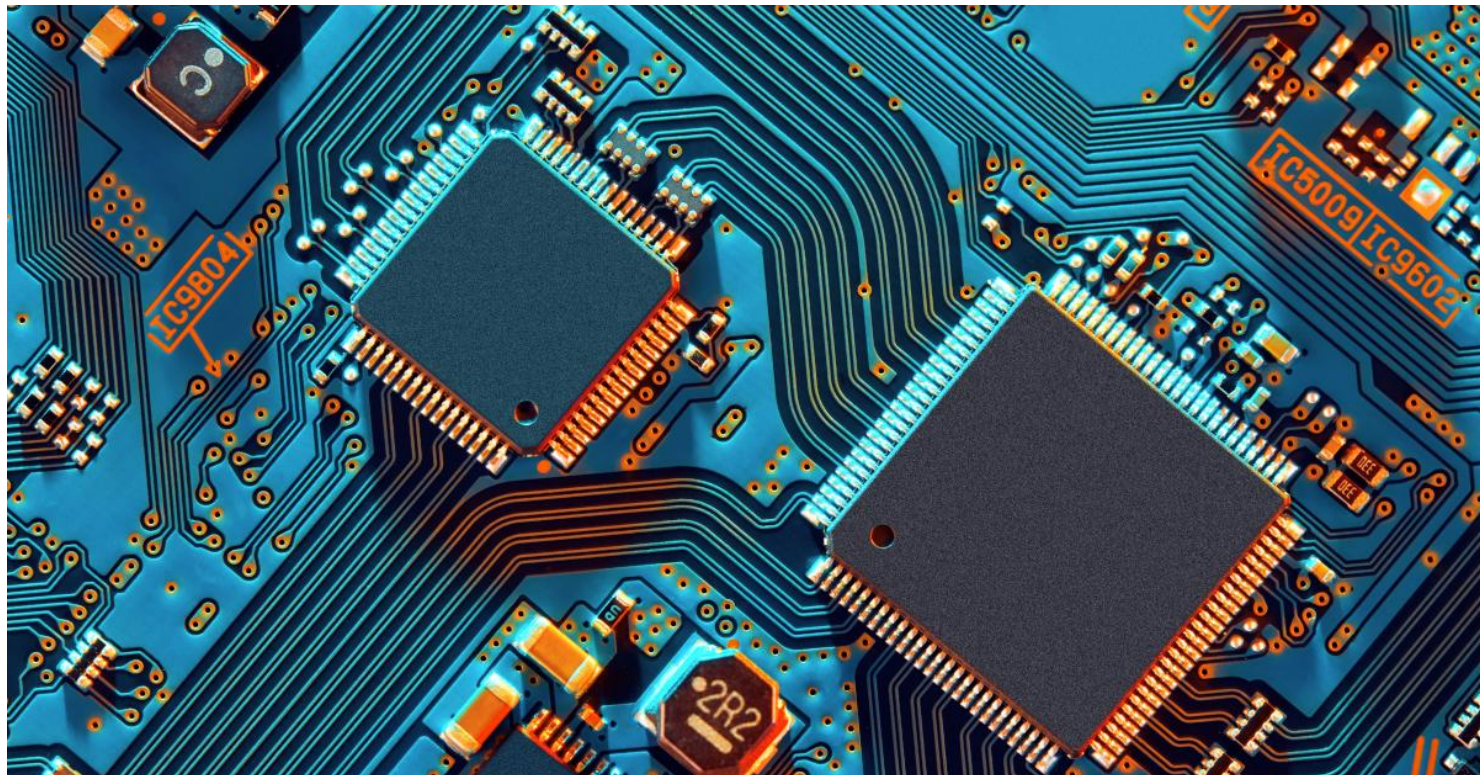




RV32I single-cycle CPU 프로젝트

조강혁

- 개요
- Block Diagram
- C Code
 - main 함수
 - sort 함수
 - swap 함수
- 결론 및 고찰

```
void main(void){
    int Num[6] = {3,5,9,1,7};
    int a=0;
    sort(Num,5);

    a=0x12345678;

    while(1);
    return;
}

void sort(int *pNum, int size){
    for(int i=0; i<size; i++){
        for(int j=0; j<size-i; j++){
            if(pNum[j] > pNum[j+1]){
                swap(&pNum[j], &pNum[j+1]);
            }
        }
    }
    return;
}

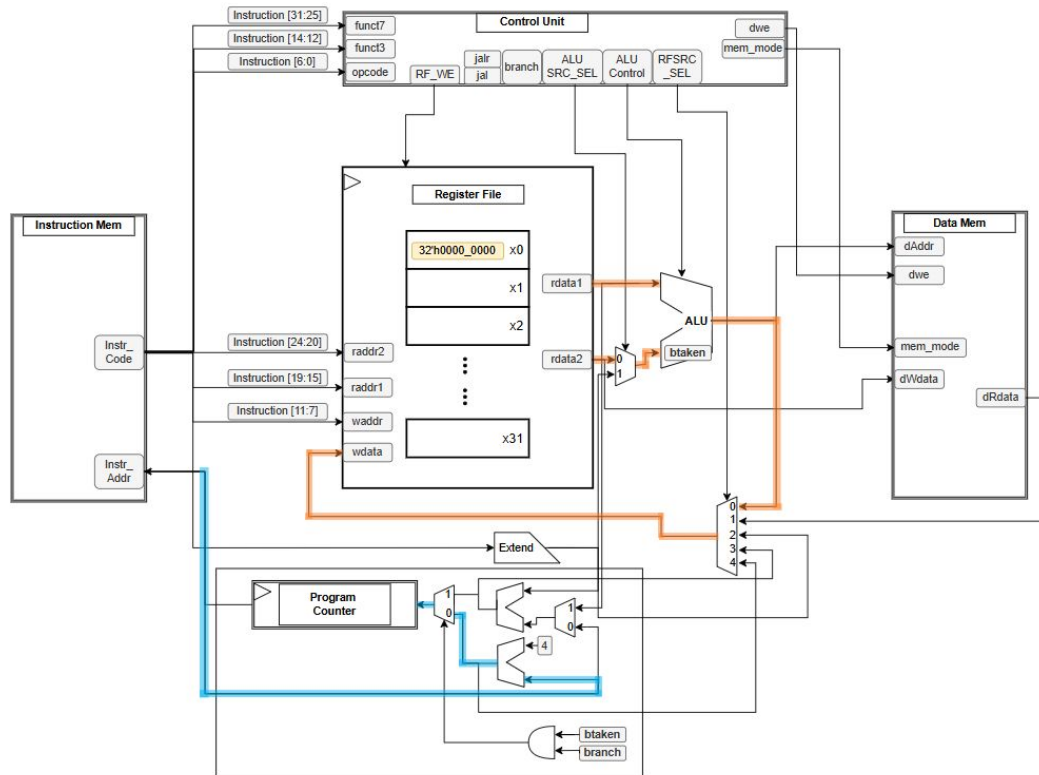
void swap(int *a, int *b){
    int temp = *a;
    *a = *b;
    *b = temp;
    return;
}
```

개요

- RV32I CPU가 Type별 Instruction에 따라 원하는 계산 및 동작을 할 수 있도록 설계한다.
- 작성된 C Code를 올려 Assembly 언어 및 Waveform 분석을 통해 해당 CPU가 오류없이 설계되었는지 확인한다.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
funct7							rs2					rs1					funct3			rd					opcode					R		
imm[12,10:5]							rs2					rs1					funct3			imm[4:1,11]					opcode					B		
imm[11:5]							rs2					rs1					funct3			imm[4:0]					opcode					S		
imm[11:0]												rs1					funct3			rd					opcode					I		
imm[11:0]												rs1					funct3			rd					opcode					IL		
imm[11:0]												rs1					funct3			rd					opcode					JL		
imm[20,10:1,11,19:12]																				rd					opcode					J		
imm[31:12]																				rd					opcode					U		

Block Diagram - R



Control Signal	
<code>rf_we</code>	1
<code>alusrc_sel</code>	0
<code>alu_control</code>	{ <code>funct[5]</code> , <code>funct3</code> }
<code>rsrc_sel</code>	3'b000
<code>mem_mode</code>	3'b000
<code>dwe</code>	0
<code>branch</code>	0
<code>jal</code>	0
<code>jalr</code>	0

funct7 rs2 rs1 funct3 rd opcode R $rd = rs1 + rs2$

Block Diagram - R

명령어 alu_control {funct7[5], funct3}

ADD 0_000 $rd = rs1 + rs2$

SUB 1_000 $rd = rs1 - rs2$

SLL 0_001 $rd = rs1 \ll rs2$

SLT 0_010 $rd = (rs1 < rs2)?1:0$

SLTU 0_011 $rd = (rs1 < rs2)?1:0$

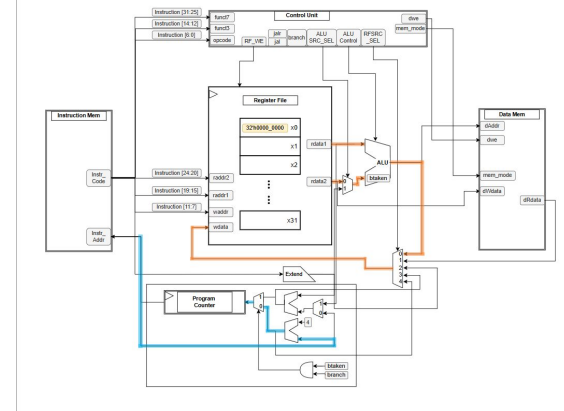
XOR 0_100 $rd = rs1 \oplus rs2$

SRL 0_101 $rd = rs1 \gg rs2$

SRA 1_101 $rd = rs1 \gg rs2$

OR 0_110 $rd = rs1 | rs2$

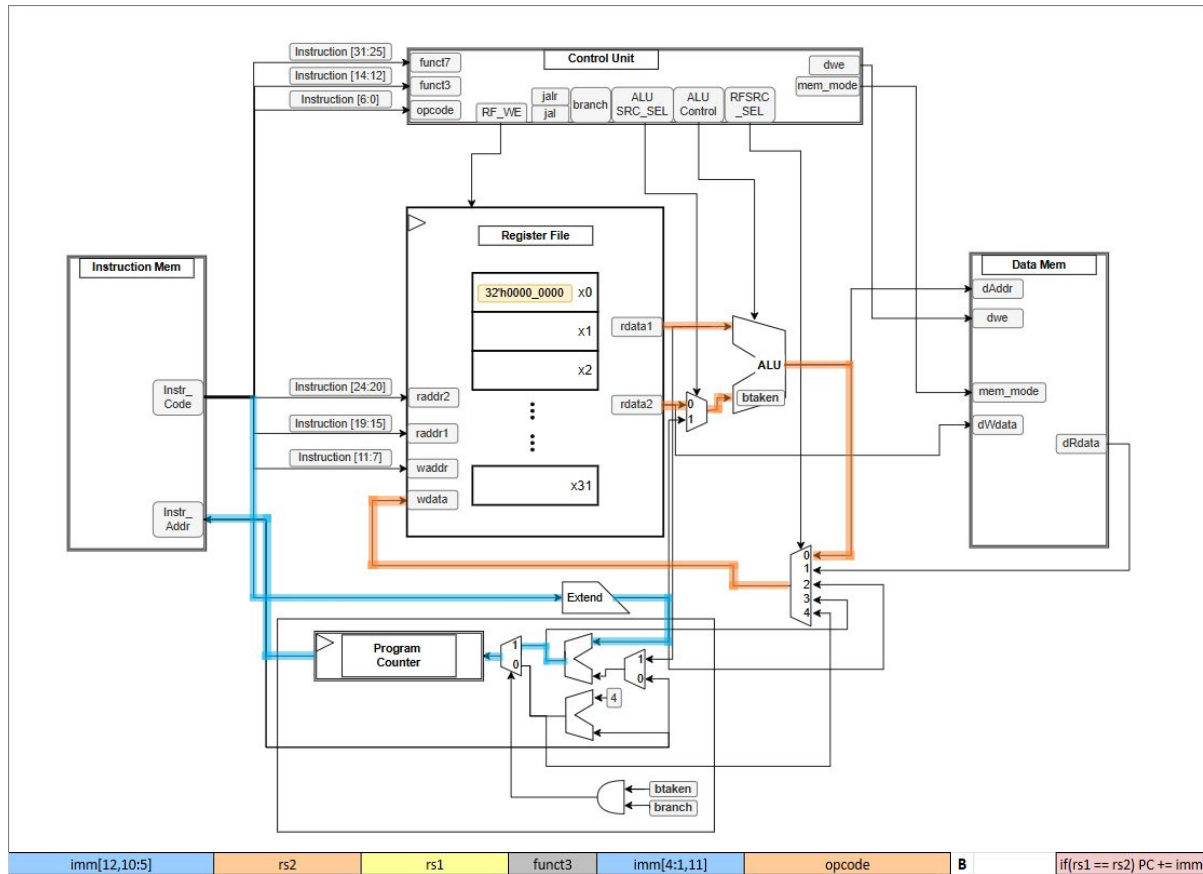
AND 0_111 $rd = rs1 \& rs2$



opcode_dbg[6:0]	DBG_IL_TYPE	DB, ...	DBG_R_TYPE
wdata[31:0]	980	980	996
alu_control[3:0]	0		0
rdata1[31:0]	976	976	980
rdata2[31:0]	X	X	16

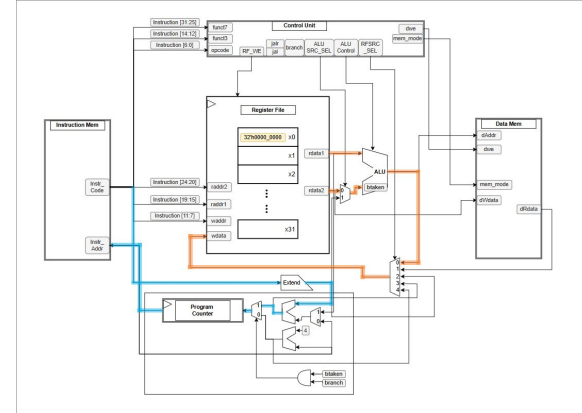


Block Diagram - B



Control Signal	
rf_we	0
alusrc_sel	0
alu_control	{1'b0, funct3}
rfsrc_sel	3'b000
mem_mode	3'b000
dwe	0
branch	1
jalr	0
jal	0

Block Diagram - B



명령어 alu_control {1'b0, funct3}

BEQ 0_000 if(rs1 == rs2) PC += imm

BNE 0_001 if(rs1 != rs2) PC += imm

BLT 0_100 if(rs1 < rs2) PC += imm

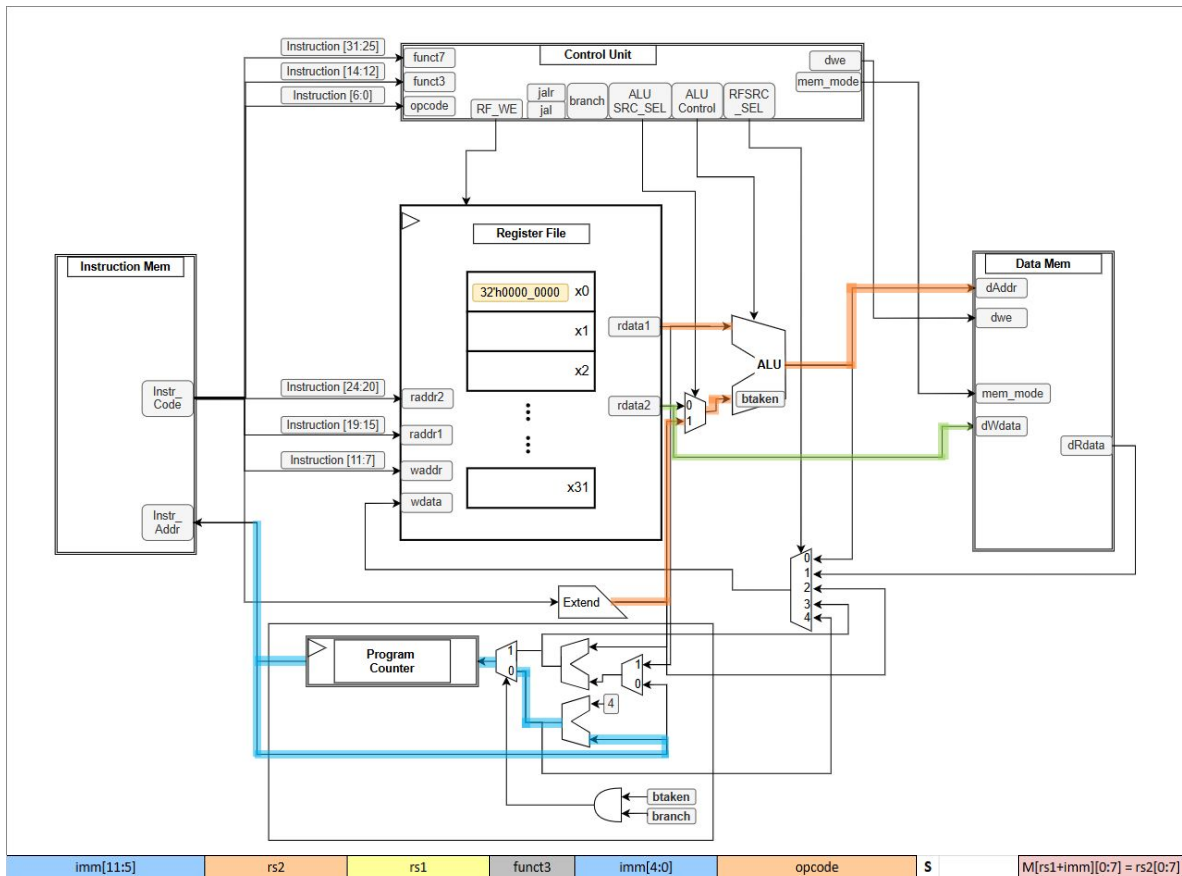
BGE 0_101 if(rs1 >= rs2) PC += imm

BLTU 0_110 if(rs1 < rs2) PC += imm

BGEU 0_111 if(rs1 >= rs2) PC += imm

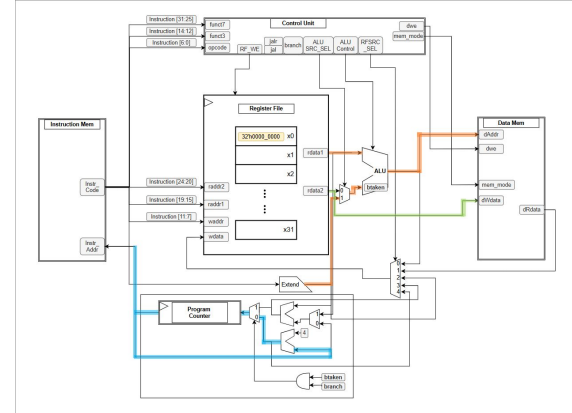
opcode_dbg[6:0]	DBG_IL_TYPE	DBG_IL_TYPE	DBG_B_TYPE	DBG_S_TYPE
mem_mode[2:0]	2	2	0	2
imm_extend[31:0]	-20	-20	-40	-156
wdata[31:0]	0	0	5	952
alu_control[3:0]	0	0	4	
rdata1[31:0]	976	976	0	976
rdata2[31:0]	X	X	5	0
pc_out[31:0]	272	300	304	308
daddr[31:0]	000003bc	000...	000003a8	00000005
				000003b8

Block Diagram - S



Control Signal	
rf_we	0
alusrc_sel	1
alu_control	'ADD
rfsrc_sel	0
mem_mode	funct3
dwe	1
branch	0
jal	0
jalr	0

Block Diagram - S



명령어		mem_mode
SB	000	$M[rs1+imm][0:7] = rs2[0:7]$
SH	001	$M[rs1+imm][0:15] = rs2[0:15]$
SW	010	$M[rs1+imm][0:31] = rs2[0:31]$

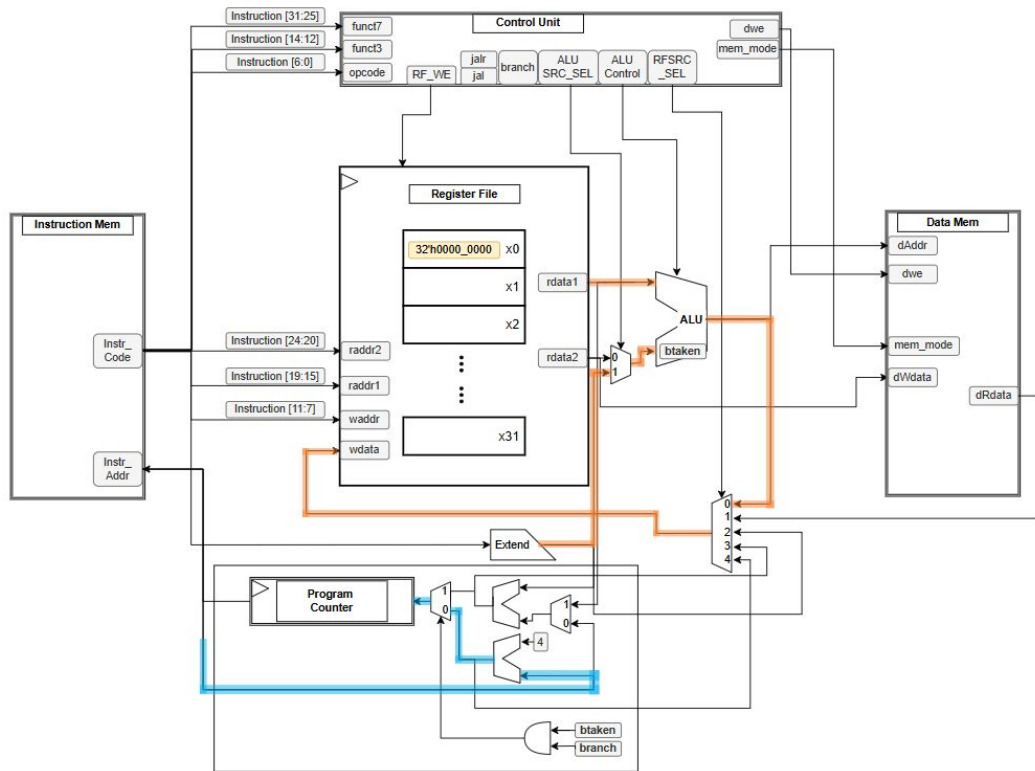
opcode_dbg[6:0]	DBG_B_TYPE	DBG...	DBG_S_TYPE
mem_mode[2:0]	000	000	010
imm_extend[31:0]	-124	-156	-24
wdata[31:0]	5	5	952
alu_control[3:0]	4	4	
rdata1[31:0]	0	0	976
rdata2[31:0]	5	5	0
pc_out[31:0]	284	308	152
daddr[31:0]	00000005	000...	000003b8

M[952]에 해당하는 M[46]에 rdata2인 0이 저장되는 것을 확인할 수 있다.

[46][31:0]	XXXXXXXX	XXXXXXXX	00000000
------------	----------	----------	----------



Block Diagram - I

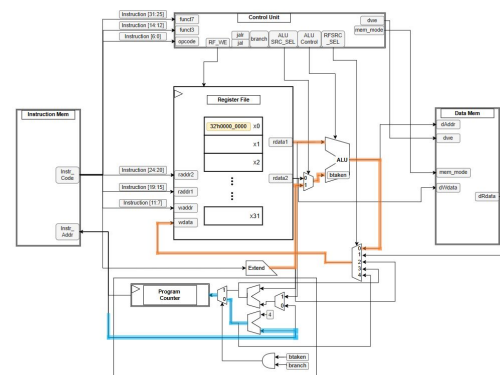


Control Signal	
rf_we	1
alusrc_sel	1
alu_control	{func7[5], func3} or {1'b0, func3}
rsrc_sel	0
mem_mode	func3
dwe	0
branch	0
jal	0
jalr	0

imm[11:0] rs1 func3 rd opcode I $rd = rs1 + imm$

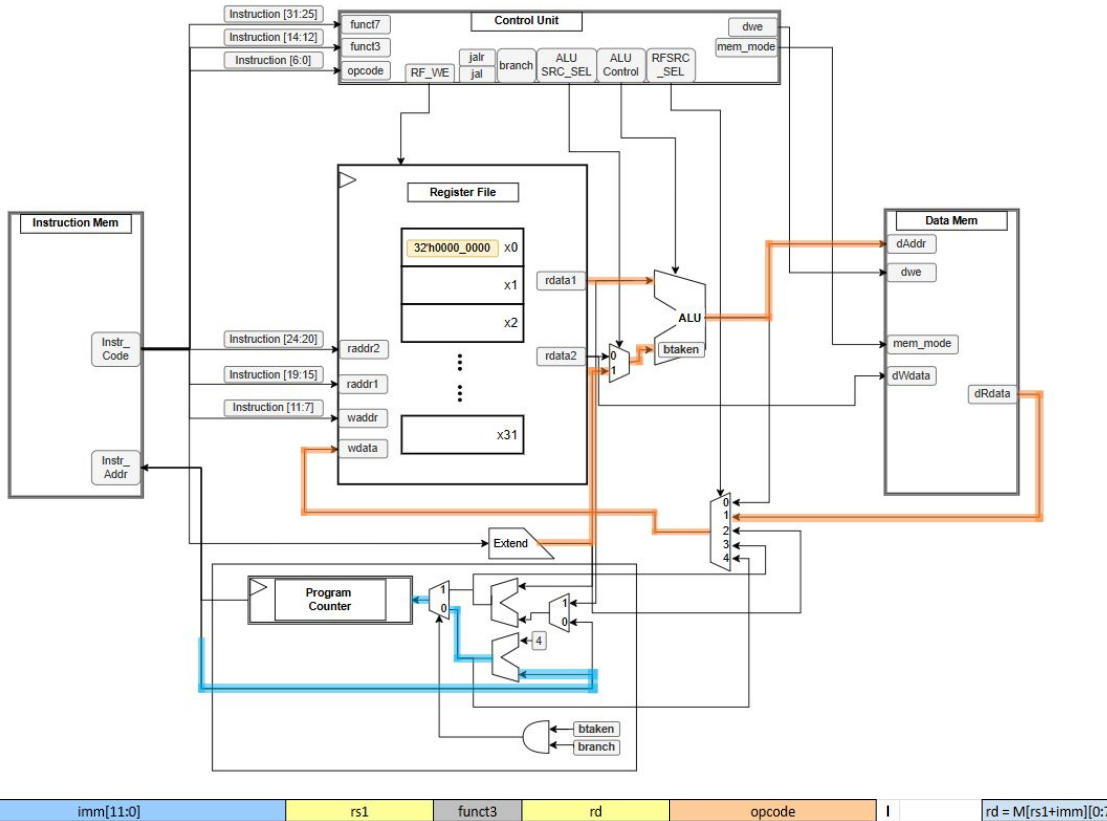


명령어	alu_control {funct[5], funct3}		
ADDI	0_000	$rd = rs1 + imm$	
SLTI	0_010	$rd = (rs1 < imm)?1:0$	
SLTIU	0_011	$rd = (rs1 < imm)?1:0$	zero-extends
XORI	0_100	$rd = rs1 \wedge imm$	
ORI	0_110	$rd = rs1 \mid imm$	
ANDI	0_111	$rd = rs1 \& imm$	
SLLI	0_001	$rd = rs1 \ll imm[0:4]$	
SRLI	0_101	$rd = rs1 \gg imm[0:4]$	
SRAI	1_101	$rd = rs1 \gg imm[0:4]$	msb-extends



opcode_dbg[6:0]	DBG_R_TYPE	DBG_IL_TYPE	DBG_I_TYPE
mem_mode[2:0]	000	010	001
imm_extend[31:0]	0	-24	2
wdata[31:0]	984	1	4
alu_control[3:0]	0	0	1
rdata1[31:0]	980	976	1
rdata2[31:0]	4	976	928
pc_out[31:0]	172	160	164

Block Diagram - IL



Control Signal

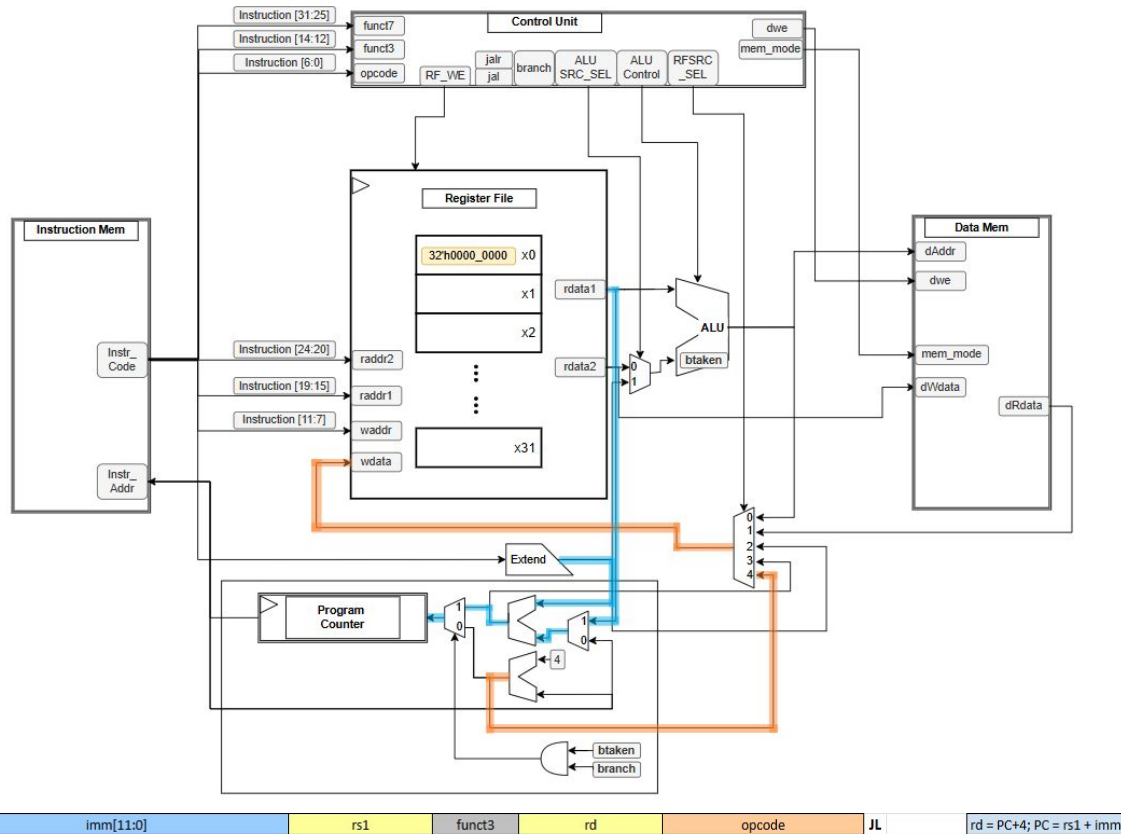
rf_we	1
alusrc_sel	1
alu_control	'ADD
rfsrc_sel	1
mem_mode	funct3
dwe	0
branch	0
jal	0
jalr	0

[illegible]

opcode_dbg[6:0]	DBG_IL_TYPE	DBG_I_TYPE	DBG_IL_TYPE
mem_mode[2:0]	010	001	010
imm_extend[31:0]	0	2	-36
wdata[31:0]	9	8	980
alu_control[3:0]	0	1	
rdata1[31:0]	988	2	976
rdata2[31:0]	0	928	X
pc_out[31:0]	200	188	192
waddr[4:0]	0f	0f	0d
daddr[31:0]	988	8	940
drdata[31:0]	00000009	XXXXXXXX	000003d4

[43][31:0]	000003d4
	000003d4

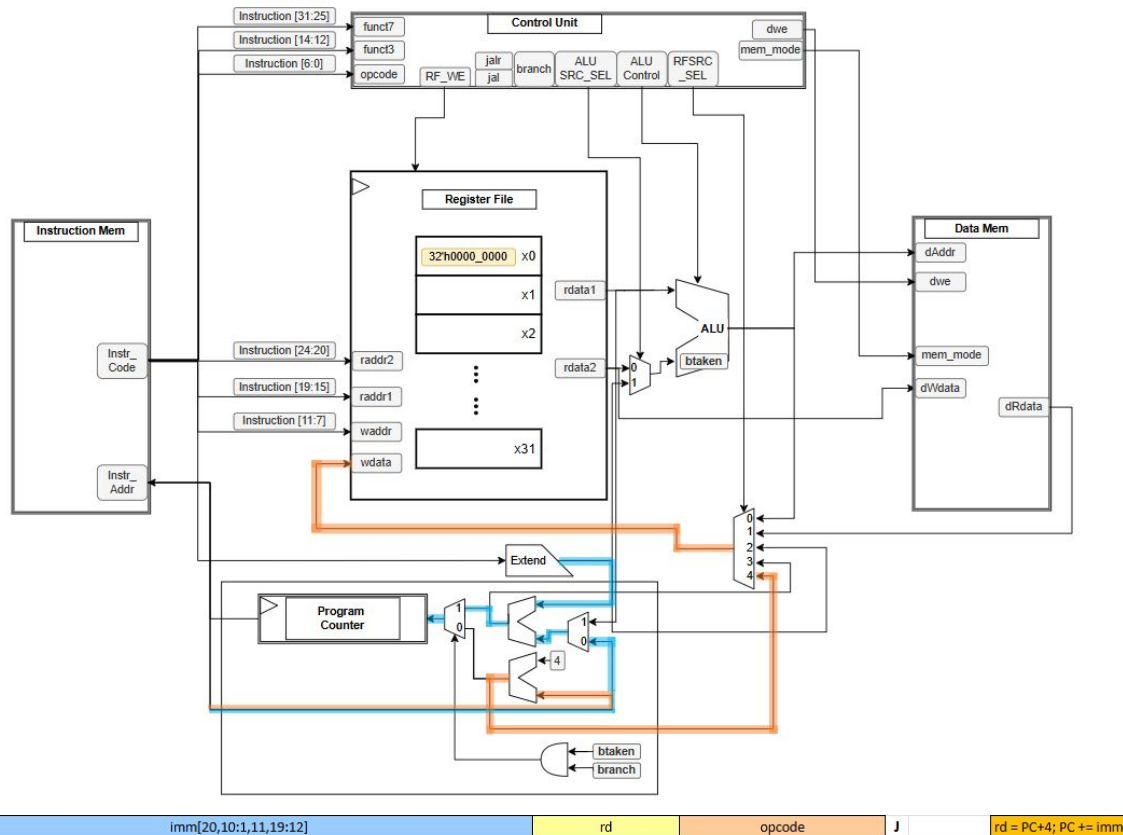
Block Diagram - JL



Control Signal

rf_we	1
alusrc_sel	0
alu_control	0
rfsrc_sel	4
mem_mode	0
dwe	0
branch	0
jal	0
jalr	1

Block Diagram - J



Control Signal

rf_we	1
alusrc_sel	0
alu_control	0
rfsrc_sel	4
mem_mode	0
dwe	0
branch	0
jalr	0

Block Diagram - J

명령어

opcode

JAL

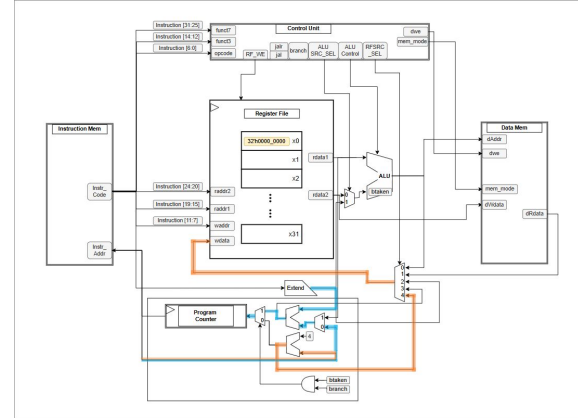
1101111

rd = PC+4; PC += imm

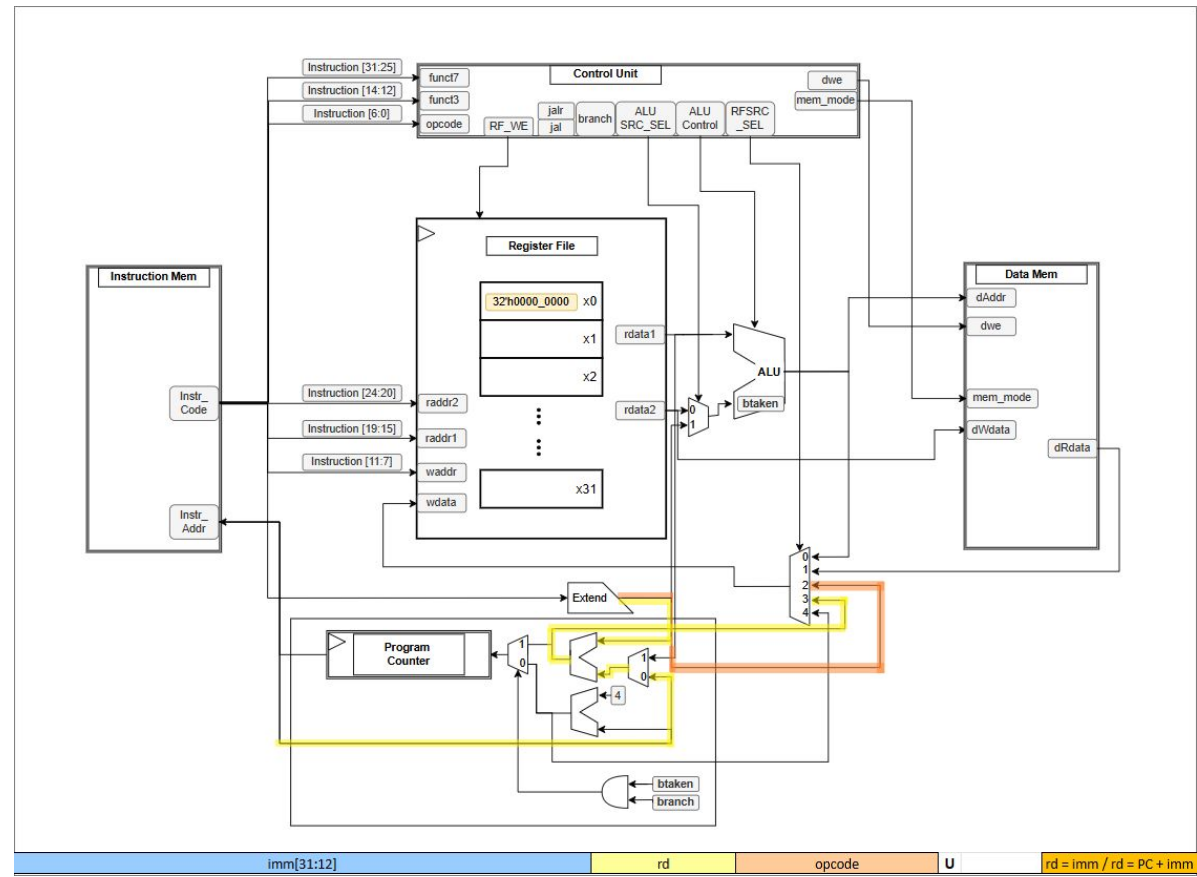
mem_mode[2:0]	000
imm_extend[31:0]	0
wdata[31:0]	2
alu_control[3:0]	8
rdata1[31:0]	5
rdata2[31:0]	3
pc_out[31:0]	276

010	000	01
-24	112	-40
952	160	5
		0
976	0	97
0		X
152	156	268

wdata는 pc+4인 160이 되고, pc가 기존 pc값 156에서 imm값 112를 더한 268이 됨을 확인할 수 있다.



Block Diagram - U



Control Signal	
rf_we	1
alusrc_sel	0
alu_control	0
rfsrc_sel	LUI = 2 / AUIPC = 3
mem_mode	0
dwe	0
branch	0
jal	0
jalr	0

명령어			opcode
LUI	011_0111	rd = imm	
AUIPC	001_0111	rd = PC + imm	

※ High-level Language

```
void sort(int *pNum, int size);
void swap(int *a, int *b);
void main(void){
    int Num[6] = {3,5,9,1,7};
    int a=0;
    sort(Num,5);

    a=0x12345678;

    while(1);
    return;
}

void sort(int *pNum, int size){
    for(int i=0; i<size; i++){
        for(int j=0; j<size-i; j++){
            if(pNum[j] > pNum[j+1]){
                swap(&pNum[j], &pNum[j+1]);
            }
        }
    }
    return;
}

void swap(int *a, int *b){
    int temp = *a;
    *a = *b;
    *b = temp;
    return;
}
```

※ Assembly Language

```
main:
    addi    sp,sp,-48
    sw      ra,44(sp)
    sw      s0,40(sp)
    addi    s0,sp,48
    sw      zero,-44(s0)
    sw      zero,-40(s0)
    sw      zero,-36(s0)
    sw      zero,-32(s0)
    sw      zero,-28(s0)
    sw      zero,-24(s0)
    li      a5,3
    sw      a5,-44(s0)
    li      a5,5
    sw      a5,-40(s0)
    li      a5,9
    sw      a5,-36(s0)
    li      a5,1
    sw      a5,-32(s0)
    li      a5,7
    sw      a5,-28(s0)
    sw      zero,-20(s0)
    addi    a5,s0,-44
    li      a1,5
    mv      a0,a5
    call    sort
    li      a5,305418240
    addi    a5,a5,1656
    sw      a5,-20(s0)
```

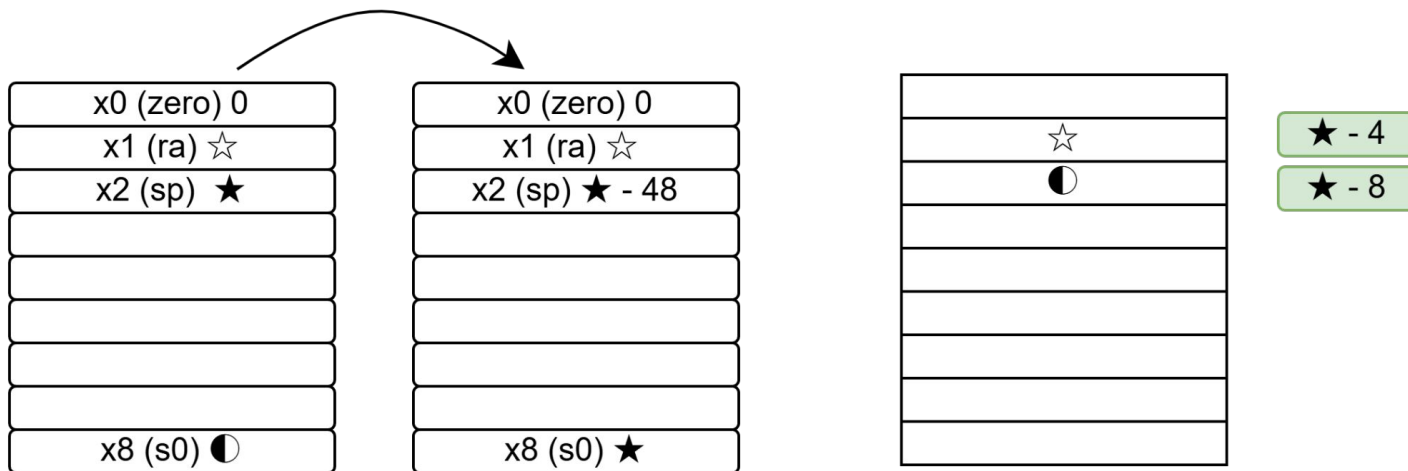
```
sort:
    addi    sp,sp,-48
    sw      ra,44(sp)
    sw      s0,40(sp)
    addi    s0,sp,48
    sw      a0,-36(s0)
    sw      a1,-40(s0)
    sw      zero,-20(s0)
    j       .L4
```

```
void swap(int *a, int *b){
    int temp = *a;
    *a = *b;
    *b = temp;
    return;
}
```

C CODE - main 함수, 프로그래밍의 시작

High-level Language	Assembly Language			
<code>void main(void) {</code>	<code>addi sp, sp, -48</code>	<code>sw ra, 44(sp)</code>	<code>sw s0, 40(sp)</code>	<code>addi s0, sp, 48</code>

※ sp: stack pointer / s0: frame pointer / ra: return address



C CODE - main 함수, 변수 설정

High-level Language	Assembly Language		
<pre>int Num[6] = {3,5,9,1,7}; a=0;</pre>	<pre>sw zero,-44(s0) sw zero,-40(s0) sw zero,-36(s0) sw zero,-32(s0) sw zero,-28(s0) sw zero,-24(s0)</pre>	<pre>li a5,3 sw a5,-44(s0) li a5,5 sw a5,-40(s0) li a5,9 sw a5,-36(s0)</pre>	<pre>li a5,1 sw a5,-32(s0) li a5,7 sw a5,-28(s0) sw zero,-20(s0)</pre>

0
0
7
1
9
5
3

★ - 20

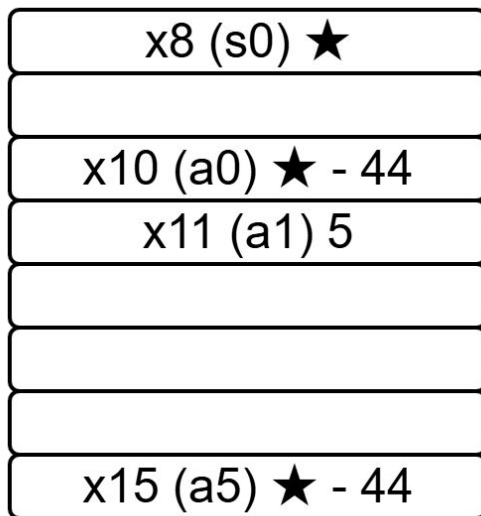
★ - 48



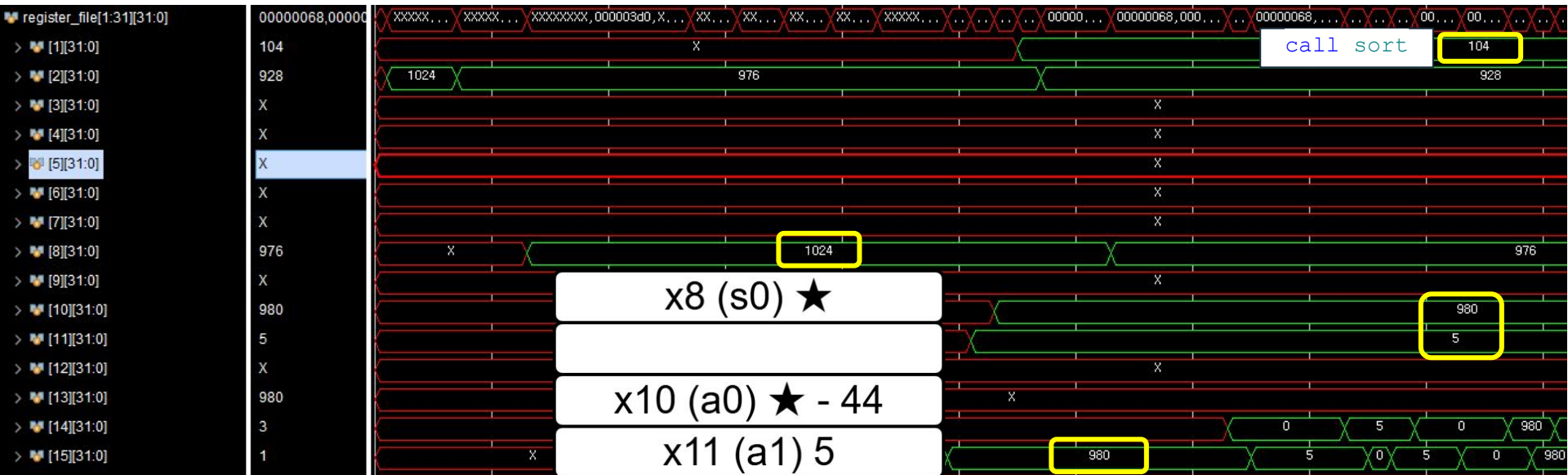
★ - 48

C CODE - main 함수, sort 인출

High-level Language	Assembly Language			
<code>sort(Num, 5);</code>	<code>addi a5, s0, -44</code>	<code>li a1, 5</code>	<code>mv a0, a5</code>	<code>call sort</code>



C CODE - main 함수, sort 인출



x8 (s0) ★

x10 (a0) ★ - 44

x11 (a1) 5

x15 (a5) ★ - 44

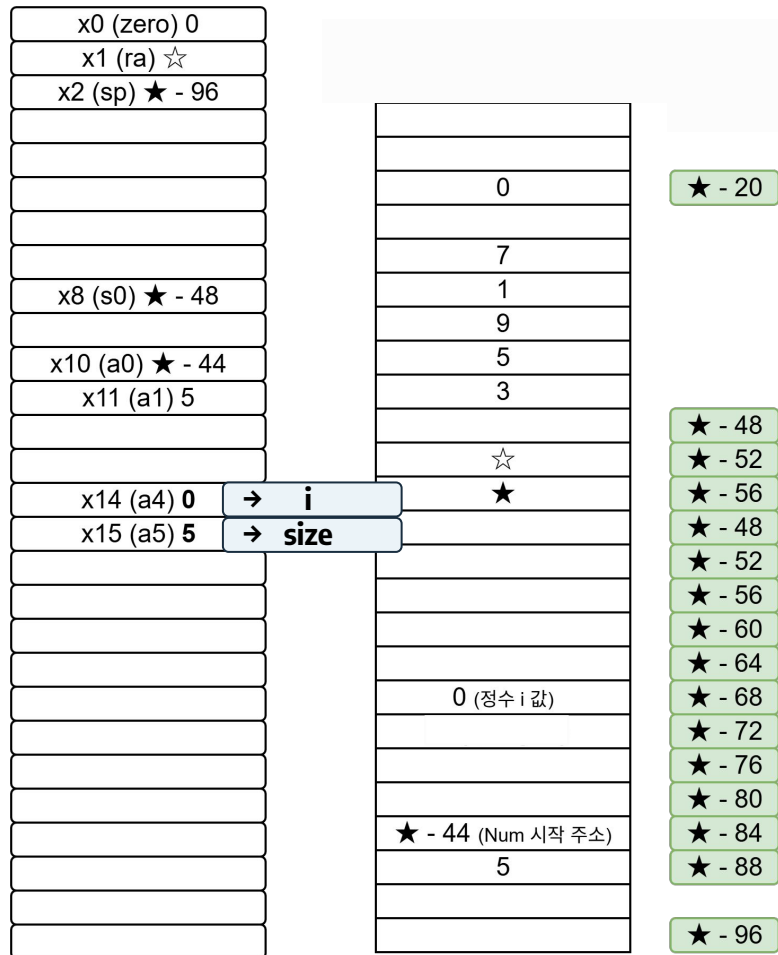
High-level Language	Assembly Language	
<pre> void sort(int *pNum, int size){ for(int i=0; i<size; i++){ for(int j=0; j<size-i; j++){ </pre>	<pre> addi sp,sp,-48 sw ra,44(sp) sw s0,40(sp) addi s0,sp,48 sw a0,-36(s0) sw a1,-40(s0) sw zero,-20(s0) j .L4 </pre>	<pre> .L4 lw a4,-20(s0) lw a5,-40(s0) blt a4,a5,.L8 .L8 sw zero,-24(s0) j .L5 .L5 lw a4,-40(s0) lw a5,-20(s0) sub a5,a4,a5 lw a4,-24(s0) blt a4,a5,.L7 </pre>

[illegible]

0	★ - 20
7	
1	
9	
5	
3	
☆	★ - 48
★	★ - 52
	★ - 56
	★ - 48
	★ - 52
	★ - 56
	★ - 60
	★ - 64
0 (정수 i 값)	★ - 68
	★ - 72
	★ - 76
	★ - 80
★ - 44 (Num 시작 주소)	★ - 84
5	★ - 88
	★ - 96

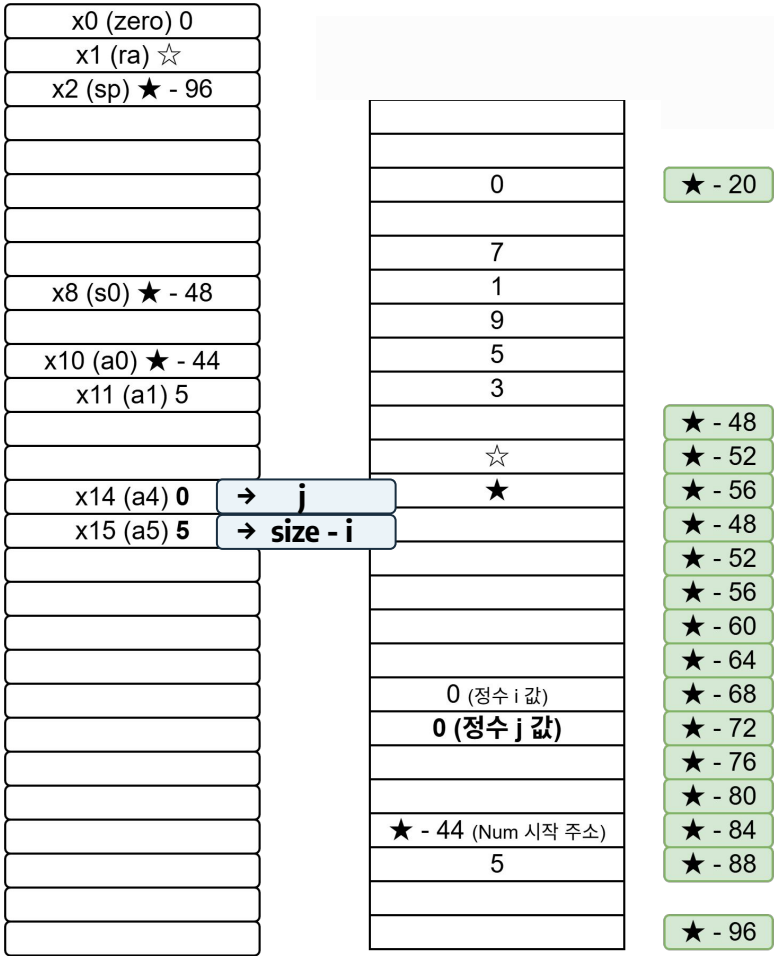
C CODE - 함수 sort 실행

High-level Language	Assembly Language	
<pre> void sort(int *pNum, int size){ for(int i=0; i<size; i++){ for(int j=0; j<size-i; j++){ </pre>	<pre> addi sp,sp,-48 sw ra,44(sp) sw s0,40(sp) addi s0,sp,48 sw a0,-36(s0) sw a1,-40(s0) sw zero,-20(s0) j .L4 </pre>	<pre> .L4 lw a4,-20(s0) lw a5,-40(s0) blt a4,a5,.L8 .L8 sw zero,-24(s0) j .L5 .L5 lw a4,-40(s0) lw a5,-20(s0) sub a5,a4,a5 lw a4,-24(s0) blt a4,a5,.L7 </pre>



C CODE - 함수 sort 실행

High-level Language	Assembly Language	
<pre>void sort(int *pNum, int size){ for(int i=0; i<size; i++){ for(int j=0; j<size-i; j++){</pre>	<pre>addi sp,sp,-48 sw ra,44(sp) sw s0,40(sp) addi s0,sp,48 sw a0,-36(s0) sw a1,-40(s0) sw zero,-20(s0) j .L4</pre>	<pre>.L4 lw a4,-20(s0) lw a5,-40(s0) blt a4,a5,.L8 .L8 sw zero,-24(s0) j .L5 .L5 lw a4,-40(s0) lw a5,-20(s0) sub a5,a4,a5 lw a4,-24(s0) blt a4,a5,.L7</pre>



1

High-level Language

```
if (pNum[j] > pNum[j+1]) {
```

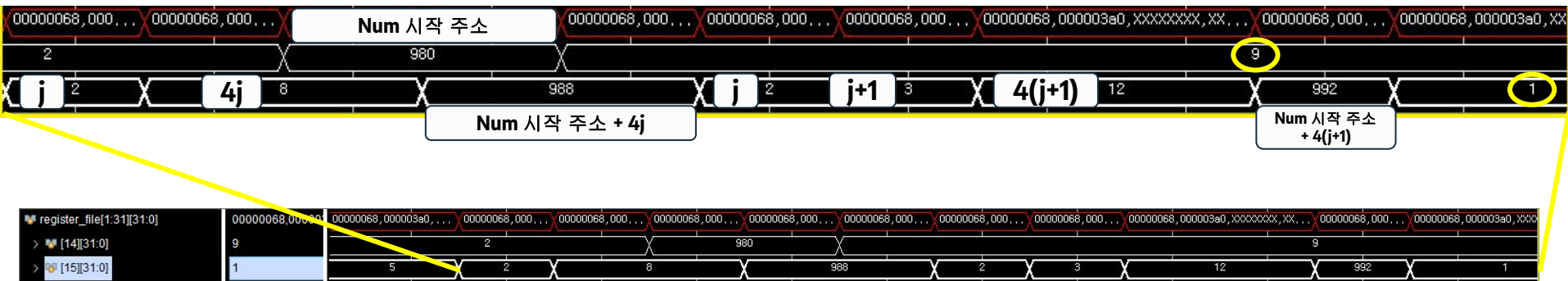
Assembly Language

High-level Language

```
if (pNum[j] > pNum[j+1]) {
```

lw	a3, -36(s0)
add	a5, a3, a5
lw	a5, 0(a5)
ble	a4, a5, <u>L6</u>
lw	a5, -24(s0)
slli	a5, a5, 2
lw	a4, -36(s0)
add	a3, a4, a5
lw	a5, -24(s0)
addi	a5, a5, 1
slli	a5, a5, 2
lw	a4, -36(s0)
add	a5, a4, a5
mv	a1, a5
mv	a0, a3
call	<u>swap</u>

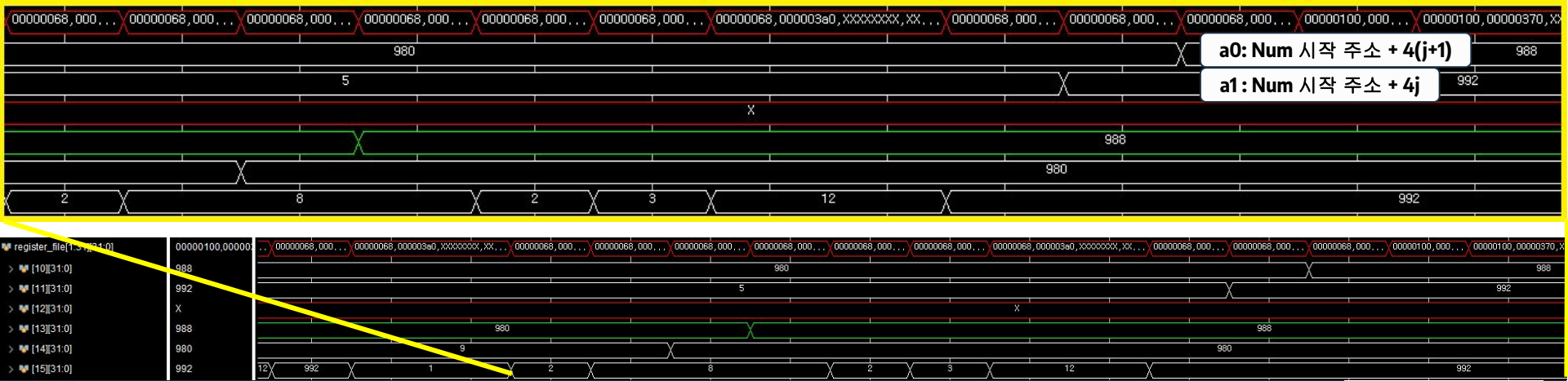
7
1
9
5
3
☆
1024
i
j
980 (Num 시작 주소)



C CODE - 함수 sort 실행

High-level Language	Assembly Language
<pre>if (pNum[j] > pNum[j+1]) {</pre>	<pre>lw a5, -24(s0) slli a5, a5, 2 lw a4, -36(s0) add a3, a4, a5 lw a5, -24(s0) addi a5, a5, 1 slli a5, a5, 2 lw a4, -36(s0) add a5, a4, a5 mv a1, a5 mv a0, a3 call swap</pre>

7	996
1	992
9	988
5	984
3	980
☆	
1024	
i	
j	s0 - 24
980 (Num 시작 주소)	s0 - 36



C CODE - 함수 swap 실행

```
void swap(int *a, int *b){  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
    return;  
}
```

swap:

```
addi    sp, sp, -48  
sw      ra, 44(sp)  
sw      s0, 40(sp)  
addi    s0, sp, 48  
sw      a0, -36(s0)  
sw      a1, -40(s0)
```

```
lw      a5, -36(s0)  
lw      a5, 0(a5)  
sw      a5, -20(s0)
```

```
lw      a5, -40(s0)  
lw      a4, 0(a5)  
lw      a5, -36(s0)  
sw      a4, 0(a5)  
lw      a5, -40(s0)  
lw      a4, -20(s0)  
sw      a4, 0(a5)
```

nop

```
lw      ra, 44(sp)  
lw      s0, 40(sp)  
addi    sp, sp, 48  
jr      ra
```

x0 (zero) 0
x1 (ra) ☆
x2 (sp) 880

x8 (s0) 928

x10 (a0) 988
x11 (a1) 992

x14 (a4) 1
x15 (a5) 9

7
1
9
5
3

996
992
988
984
980

⋮

9
988
992

s0-20 (908)

s0-36 (892)
s0-40 (888)



C CODE - 함수 swap 실행

```
void swap(int *a, int *b){  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
    return;  
}
```

swap:

```
addi    sp, sp, -48  
sw      ra, 44(sp)  
sw      s0, 40(sp)  
addi    s0, sp, 48  
sw      a0, -36(s0)  
sw      a1, -40(s0)  
lw      a5, -36(s0)  
lw      a5, 0(a5)  
sw      a5, -20(s0)  
lw      a5, -40(s0)  
lw      a4, 0(a5)  
lw      a5, -36(s0)  
sw      a4, 0(a5)  
lw      a5, -40(s0)  
lw      a4, -20(s0)  
sw      a4, 0(a5)  
nop  
lw      ra, 44(sp)  
lw      s0, 40(sp)  
addi    sp, sp, 48  
jr      ra
```

x0 (zero) 0
x1 (ra) ☆
x2 (sp) 880
x8 (s0) 928
x10 (a0) 988
x11 (a1) 992
x14 (a4) 1
x15 (a5) 992 → 988

7
1
9 → 1
5
3
⋮
9
988
992

996
992
988
984
980

s0-20 (908)

s0-36 (892)

s0-40 (888)



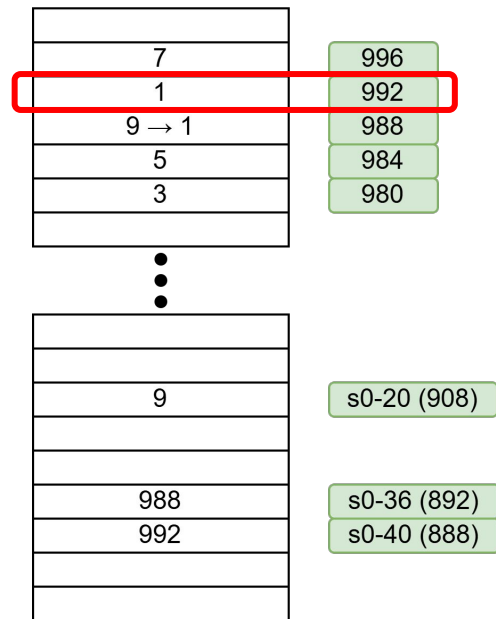
C CODE - 함수 swap 실행

```
void swap(int *a, int *b){  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
    return;  
}
```

swap:

```
addi    sp, sp, -48  
sw      ra, 44(sp)  
sw      s0, 40(sp)  
addi    s0, sp, 48  
sw      a0, -36(s0)  
sw      a1, -40(s0)  
lw      a5, -36(s0)  
lw      a5, 0(a5)  
sw      a5, -20(s0)  
lw      a5, -40(s0)  
lw      a4, 0(a5)  
lw      a5, -36(s0)  
sw      a4, 0(a5)  
lw      a5, -40(s0)  
lw      a4, -20(s0)  
sw      a4, 0(a5)  
nop  
lw      ra, 44(sp)  
lw      s0, 40(sp)  
addi    sp, sp, 48  
jr      ra
```

x0 (zero) 0
x1 (ra) ☆
x2 (sp) 880
x8 (s0) 928
x10 (a0) 988
x11 (a1) 992
x14 (a4) 1
x15 (a5) 992 → 988



C CODE - 함수 swap 실행

```
void swap(int *a, int *b){  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
    return;  
}
```

swap:

```
addi    sp, sp, -48  
sw      ra, 44(sp)  
sw      s0, 40(sp)  
addi    s0, sp, 48  
sw      a0, -36(s0)  
sw      a1, -40(s0)  
lw      a5, -36(s0)  
lw      a5, 0(a5)  
sw      a5, -20(s0)  
lw      a5, -40(s0)  
lw      a4, 0(a5)  
lw      a5, -36(s0)  
sw      a4, 0(a5)  
lw      a5, -40(s0)  
lw      a4, -20(s0)  
sw      a4, 0(a5)  
nop  
lw      ra, 44(sp)  
lw      s0, 40(sp)  
addi    sp, sp, 48  
jr      ra
```

x0 (zero) 0
x1 (ra) ☆
x2 (sp) 880
x8 (s0) 928
x10 (a0) 988
x11 (a1) 992
x14 (a4) 1
x15 (a5) 992 → 988

7
1
9 → 1
5
3
⋮
9
988
992

996
992
988
984
980

s0-20 (908)

s0-36 (892)

s0-40 (888)



C CODE - 함수 swap 실행

```
void swap(int *a, int *b){  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
    return;  
}
```

swap:

```
addi    sp, sp, -48  
sw      ra, 44(sp)  
sw      s0, 40(sp)  
addi    s0, sp, 48  
sw      a0, -36(s0)  
sw      a1, -40(s0)  
lw      a5, -36(s0)  
lw      a5, 0(a5)  
sw      a5, -20(s0)  
lw      a5, -40(s0)  
lw      a4, 0(a5)  
lw      a5, -36(s0)  
sw      a4, 0(a5)  
lw      a5, -40(s0)  
lw      a4, -20(s0)  
sw      a4, 0(a5)  
nop  
lw      ra, 44(sp)  
lw      s0, 40(sp)  
addi    sp, sp, 48  
jr      ra
```

x0 (zero) 0
x1 (ra) ☆
x2 (sp) 880
x8 (s0) 928
x10 (a0) 988
x11 (a1) 992
x14 (a4) 1
x15 (a5) 992 → 988

7	996
1	992
9 → 1	988
5	984
3	980
⋮	
9	s0-20 (908)
988	s0-36 (892)
992	s0-40 (888)

C CODE - 함수 swap 실행

```
void swap(int *a, int *b){  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
    return;  
}
```

swap:

```
addi    sp, sp, -48  
sw      ra, 44(sp)  
sw      s0, 40(sp)  
addi    s0, sp, 48  
sw      a0, -36(s0)  
sw      a1, -40(s0)  
lw      a5, -36(s0)  
lw      a5, 0(a5)  
sw      a5, -20(s0)  
lw      a5, -40(s0)  
lw      a4, 0(a5)  
lw      a5, -36(s0)  
sw      a4, 0(a5)  
lw      a5, -40(s0)  
lw      a4, -20(s0)  
sw      a4, 0(a5)  
nop  
lw      ra, 44(sp)  
lw      s0, 40(sp)  
addi    sp, sp, 48  
jr      ra
```

x0 (zero) 0
x1 (ra) ☆
x2 (sp) 880
x8 (s0) 928
x10 (a0) 988
x11 (a1) 992
x14 (a4) 9
x15 (a5) 988 → 992

7
1 → 9
1
5
3
⋮
9
988
992

996
992
988
984
980

s0-20 (908)
s0-36 (892)
s0-40 (888)

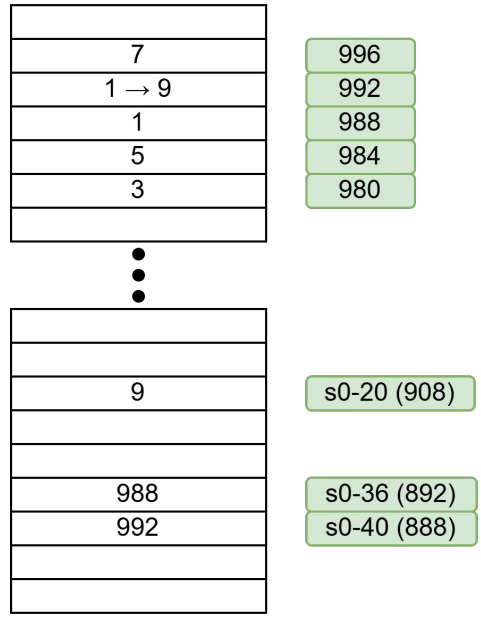


C CODE - 함수 swap 실행

```
void swap(int *a, int *b){
    int temp = *a;
    *a = *b;
    *b = temp;
    return;
}
```

```
swap:
    addi    sp,sp,-48
    sw      ra,44(sp)
    sw      s0,40(sp)
    addi    s0,sp,48
    sw      a0,-36(s0)
    sw      a1,-40(s0)
    lw      a5,-36(s0)
    lw      a5,0(a5)
    sw      a5,-20(s0)
    lw      a5,-40(s0)
    lw      a4,0(a5)
    lw      a5,-36(s0)
    sw      a4,0(a5)
    lw      a5,-40(s0)
    lw      a4,-20(s0)
    sw      a4,0(a5)
    nop
    lw      ra,44(sp)
    lw      s0,40(sp)
    addi    sp,sp,48
    jr      ra
```

x0 (zero) 0
x1 (ra) ☆
x2 (sp) 880
x8 (s0) 928
x10 (a0) 988
x11 (a1) 992
x14 (a4) 9
x15 (a5) 988 → 992



C CODE - 함수 swap 실행

```
void swap(int *a, int *b){  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
    return;  
}
```

swap:

```
addi    sp, sp, -48  
sw      ra, 44(sp)  
sw      s0, 40(sp)  
addi    s0, sp, 48  
sw      a0, -36(s0)  
sw      a1, -40(s0)  
lw      a5, -36(s0)  
lw      a5, 0(a5)  
sw      a5, -20(s0)  
lw      a5, -40(s0)  
lw      a4, 0(a5)  
lw      a5, -36(s0)  
sw      a4, 0(a5)  
lw      a5, -40(s0)  
lw      a4, -20(s0)  
sw      a4, 0(a5)  
nop  
lw      ra, 44(sp)  
lw      s0, 40(sp)  
addi    sp, sp, 48  
jr      ra
```

x0 (zero) 0
x1 (ra) ☆
x2 (sp) 880
x8 (s0) 928
x10 (a0) 988
x11 (a1) 992
x14 (a4) 9
x15 (a5) 988 → 992

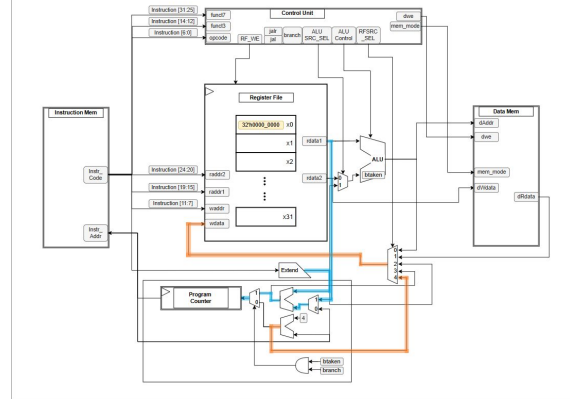
	7	996
	1 → 9	992
	1	988
	5	984
	3	980
	⋮	
	9	s0-20 (908)
	988	s0-36 (892)
	992	s0-40 (888)

C CODE - 함수 swap 실행

```
void swap(int *a, int *b){  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
    return;  
}
```

swap:

```
addi    sp,sp,-48  
sw      ra,44(sp)  
sw      s0,40(sp)  
addi    s0,sp,48  
sw      a0,-36(s0)  
sw      a1,-40(s0)  
lw      a5,-36(s0)  
lw      a5,0(a5)  
sw      a5,-20(s0)  
lw      a5,-40(s0)  
lw      a4,0(a5)  
lw      a5,-36(s0)  
sw      a4,0(a5)  
lw      a5,-40(s0)  
lw      a4,-20(s0)  
sw      a4,0(a5)  
nop  
lw      ra,44(sp)  
lw      s0,40(sp)  
addi    sp,sp,48  
jr      ra
```



00008067

000000000000_00001_000_00000_110_0111

imm[11:0] = 0

rs1 = 1 (x1)

funct3 = 0

rd = 0 (x0)

opcode = 110_0111

rd = PC + 4; PC = rs1 + imm

1. X0에 PC + 4를 저장하기에 RA를 저장하지 않겠다는 의미

2. imm = 0이므로 rs1, 즉 저장되어있던 RA로 돌아가겠다는 의미



C CODE - 함수 swap 실행

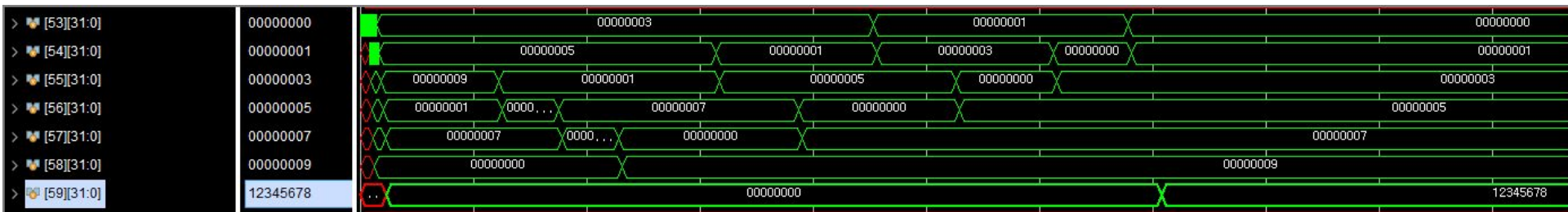
```
void sort(int *pNum, int size){  
    for(int i=0; i<size; i++){  
        for(int j=0; j<size-i; j++){  
            if(pNum[j] > pNum[j+1]){  
                swap(&pNum[j], &pNum[j+1]);  
            }  
        }  
    }  
    return;  
}
```

```
.L7:  
    lw     a5, -24(s0)  
    slli   a5, a5, 2  
    lw     a4, -36(s0)  
    add    a5, a4, a5  
    lw     a4, 0(a5)  
    lw     a5, -24(s0)  
    addi   a5, a5, 1  
    slli   a5, a5, 2  
    lw     a3, -36(s0)  
    add    a5, a3, a5  
    lw     a5, 0(a5)  
    ble    a4, a5, .L6  
    lw     a5, -24(s0)  
    slli   a5, a5, 2  
    lw     a4, -36(s0)  
    add    a3, a4, a5  
    lw     a5, -24(s0)  
    addi   a5, a5, 1  
    slli   a5, a5, 2  
    lw     a4, -36(s0)  
    add    a5, a4, a5  
    mv     a1, a5  
    mv     a0, a3  
    call   swap
```

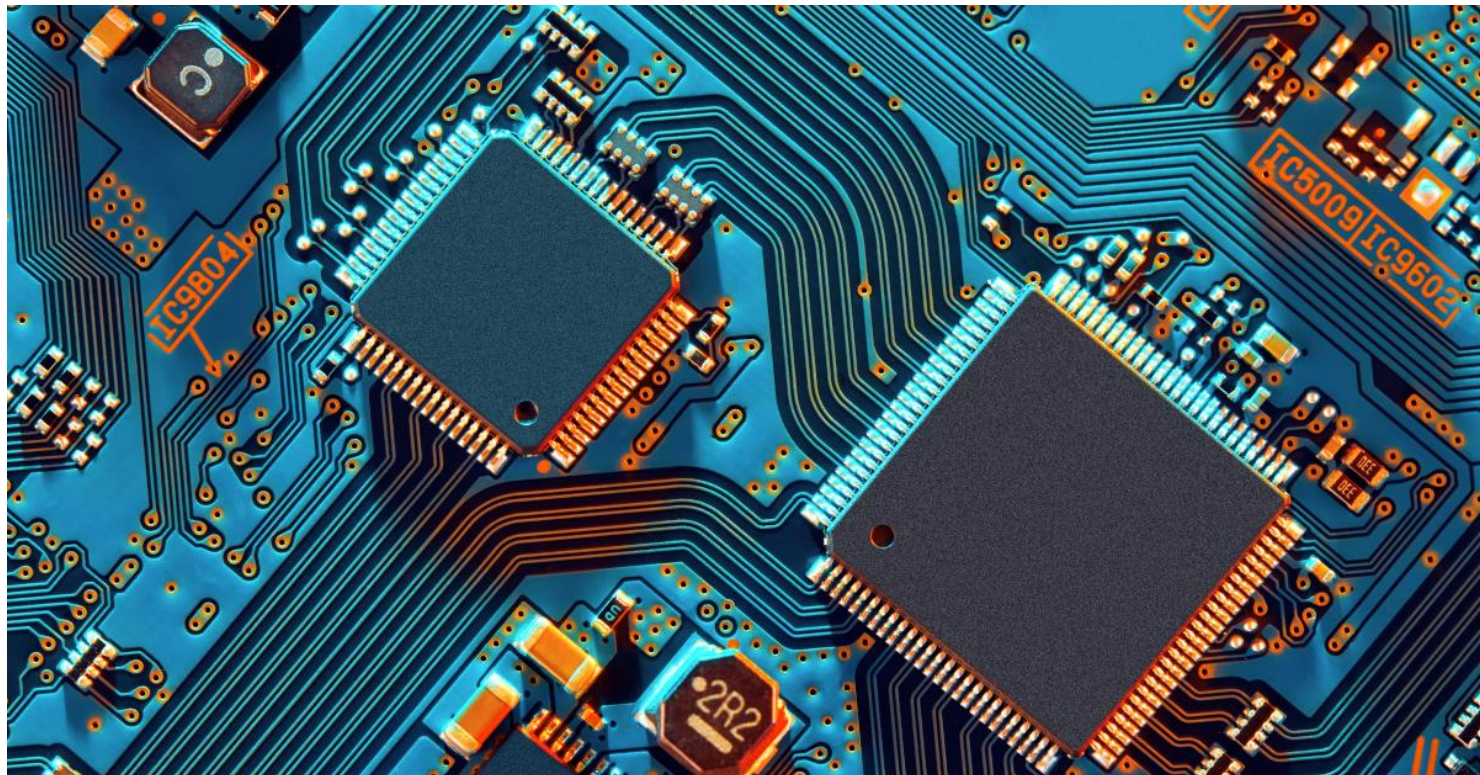
```
.L6:  
    lw     a5, -24(s0)  
    addi   a5, a5, 1  
    sw     a5, -24(s0)
```

return address : pc + 4

9	996
7	992
5	988
3	984
1	980



- 함수 시작 시 RA, s0, SP의 작동 방식을 파악할 수 있었다.
- 변수 설정 시, Memory에 할당되는 방식과 어떻게 사용되는지 확인할 수 있었다.
- Type별 Instruction에 대한 CPU 작동 방식을 파악하고, Assembly 언어와 익숙해질 수 있었다.



감사합니다.
Thank You